CẨM NANG NGÀNH DEVOPS

Giáo trình kỹ thuật · bản đọc thử 3 chương

Chưa phải bản phát hành hoàn chỉnh

MỤC LỤC

Chương 1. Từ chương trình đến tiến trình 3

1.1. Ba lớp thường bị gọi chung là “ứng dụng” . . . 3

1.2. Một tiến trình ra đời và đổi chương trình . . . 3

1.3. Tên tệp, file descriptor và đối tượng đang mở . . . 4

1.4. Quyền và giới hạn: tiến trình không có toàn quyền chỉ vì nó chạy . . . 5

1.5. Từ tiến trình đến dịch vụ . . . 5

1.6. Áp mô hình vào một sự cố nhỏ . . . 6

Bài tập . . . 6

Chương 2. CPU, bộ nhớ và I/O: thời gian đi đâu? 7

2.1. Công việc, thời gian chạy và thời gian chờ . . . 7

2.2. Địa chỉ ảo không phải RAM đã dùng . . . 7

2.3. Page cache và lời hứa ghi dữ liệu . . . 8

2.4. Giới hạn tài nguyên và hành vi dưới tải . . . 8

2.5. Lần theo một phút tăng độ trễ . . . 9

Bài tập . . . 10

Chương 3. Nhân Linux, hệ thống tệp, quyền và systemd 11

3.1. Từ mã người dùng đến lời gọi hệ thống . . . 11

3.2. Tên tệp, inode và bước đi qua thư mục . . . 11

3.3. Một unit biến tiến trình thành dịch vụ như thế nào . . . 12

3.4. Kiểm chứng trước và sau khi khởi động . . . 13

Bài tập . . . 14

Lời giải 15

Chương 1. Từ chương trình đến tiến trình . . . 15

Chương 2. CPU, bộ nhớ và I/O: thời gian đi đâu? . . . 15

Chương 3. Nhân Linux, hệ thống tệp, quyền và systemd . . . 16

Chương 1. Từ chương trình đến tiến trình

Một dịch vụ có thể có mã nguồn đúng mà vẫn không phục vụ được yêu cầu. Tệp thực thi có thể tồn tại, nhưng chưa có tiến trình nào đang chạy; tiến trình có thể chạy, nhưng không mở được cổng; cổng có thể mở, nhưng tiến trình không đọc được tệp cấu hình. Để chẩn đoán những tình huống này, trước hết cần biết hệ điều hành đã biến chương trình thành một thực thể đang hoạt động như thế nào. Chương này xây dựng mô hình ấy trước khi nói đến container, điều phối hay giám sát.

1.1. Ba lớp thường bị gọi chung là “ứng dụng”

Mã nguồn là văn bản mô tả hành vi. Tệp thực thi hoặc chương trình được thông dịch là vật liệu để khởi chạy. Tiến trình là một lần thực thi cụ thể: nó có định danh, không gian địa chỉ, trạng thái xử lý, thông tin định danh người dùng và tập tài nguyên đang mở. Một chương trình có thể được chạy nhiều lần, tạo ra nhiều tiến trình độc lập. Ngược lại, một tiến trình có thể thay chương trình đang thực thi mà vẫn giữ định danh tiến trình, khi dùng cơ chế `execve`.

Sự phân biệt này giải thích vì sao “build thành công” không đồng nghĩa “dịch vụ sống”. Build xác nhận một số tính chất của artifact; nó không chứng minh hệ điều hành cấp đủ tài nguyên, địa chỉ mạng sẵn có hay cấu hình runtime hợp lệ. Đối với người vận hành, câu hỏi cần tách làm hai: artifact nào được đưa tới máy, và tiến trình nào đang thực thi artifact đó? Nếu hai câu trả lời khác nhau, một lần triển khai có thể đã thay tệp trên đĩa mà chưa thay phiên bản đang phục vụ.

CPU thực thi lệnh của tiến trình theo các khoảng thời gian; bộ nhớ chứa mã và dữ liệu cần cho lần chạy; nhân hệ điều hành quản lý tài nguyên và cung cấp các lời gọi hệ thống. Đó là mô hình khởi đầu, chưa phải toàn bộ chi tiết của Linux. Trong một tiến trình còn có thể có nhiều luồng thực thi cùng chia sẻ phần lớn không gian địa chỉ. Vì thế, số tiến trình không phải số công việc đồng thời, và mức dùng CPU không thể suy ra chỉ từ số PID.

1.2. Một tiến trình ra đời và đổi chương trình

Trên Linux, `fork` tạo tiến trình con. Tiến trình cha nhận PID của con, còn con nhận số 0; nếu không tạo được con, chỉ cha nhận `-1` và mã lỗi. Nhờ vậy hai tiến trình đi theo hai nhánh xử lý khác nhau. Không nên hình dung rằng toàn bộ bộ nhớ vật lý bị sao chép ngay lập tức: triển khai Linux dùng các trang copy-on-write, tạo bản riêng khi cần ghi. Đây là chi tiết triển khai Linux, không phải định nghĩa phổ quát của mọi hệ điều hành.

Sau bước tạo con, một cách khởi chạy chương trình khác là gọi `execve`. Lời gọi này thay process image hiện tại bằng chương trình mới; nó không tạo thêm một tiến trình mới. Nếu thành công, luồng lệnh cũ không tiếp tục ngay sau lời gọi. Nếu lỗi, lời gọi trả về `-1`, đặt `errno` và chương trình cũ còn cơ hội xử lý. Cặp cơ chế tạo con rồi thay chương trình giúp hiểu vì sao một shell có thể tiếp tục tồn tại sau khi khởi chạy lệnh khác, đồng thời vì sao tiến trình của lệnh có PID riêng.

Ví dụ chạy trên Linux bằng `python3 fork_exec.py`; thêm `fail` để cố ý chọn đường dẫn không tồn tại. PID do hệ điều hành cấp. Cha in trạng thái sau khi `waitpid` trả về.

```
"""Linux-only trace of fork and execve; run with optional argument 'fail'."""

import os
import sys


child = os.fork()
if child == 0:
    path = "/definitely/missing" if sys.argv[1:] == ["fail"] else "/bin/echo"
    os.write(2, f"child: fork returned {child}, PID {os.getpid()}\n".encode())
    try:
        os.execve(path, ["echo", "child: execve succeeded"], os.environ.copy())
    except OSError as exc:
        os.write(2, f"child: execve failed: {exc.strerror}\n".encode())
        os._exit(127)

print(f"parent: fork returned child PID {child}", flush=True)
_, status = os.waitpid(child, 0)
if os.WIFEXITED(status):
    print(f"parent: child exited with {os.WEXITSTATUS(status)}")
else:
    print("parent: child did not exit normally")
```

Ở cả hai lần chạy, `fork` thành công tạo hai PID. Trong cha, `child` là PID dương của con; trong con, `child` bằng 0. Nhánh thành công thay chương trình của con bằng `/bin/echo`, giữ nguyên PID của con và không vào khối `except`. Nhánh lỗi khiến lời gọi Python ném `OSError`; con in lỗi rồi thoát với mã 127 theo quy ước riêng của ví dụ, còn cha nhận mã ấy qua `waitpid`. Ở giao diện lời gọi hệ thống C, lỗi tương ứng là giá trị `-1` và `errno`; Python chuyển nó thành ngoại lệ. Nếu `fork` thất bại, Python cũng ném `OSError` và không tạo con để chờ. Mã 127 ở đây là lựa chọn của chương trình, không phải mã lỗi cố định do `execve` áp đặt.

Ví dụ chỉ dành cho tiến trình Python một luồng. Từ Python 3.12, `os.fork()` có thể phát `DeprecationWarning` nếu phát hiện tiến trình có nhiều luồng: các khóa do luồng khác giữ có thể khiến tiến trình con kẹt trước khi thay chương trình. Không dùng đoạn mã này làm mẫu khởi chạy trong ứng dụng Python đa luồng.

Mô hình này cũng giải thích một lỗi vận hành phổ biến: chương trình con kế thừa tài nguyên mà người gọi quên đóng. Theo mặc định, một file descriptor có thể còn mở qua `execve`; cờ close-on-exec thay đổi hành vi đó. Nếu vô tình giữ đầu ghi của một pipe ở tiến trình khác, đầu đọc có thể chưa nhìn thấy kết thúc luồng dù bên tạo dữ liệu đã xong. Kết luận này là hệ quả của vòng đời tài nguyên, không phải một quy tắc riêng của Bash.

1.3. Tên tệp, file descriptor và đối tượng đang mở

Khi tiến trình gọi `open`, nó nhận một số nguyên nhỏ không âm: file descriptor. Số này là chỉ số trong bảng descriptor của chính tiến trình. Descriptor không phải tên tệp, cũng không phải bản thân nội dung tệp. Nó tham chiếu tới một open file description do nhân quản lý, nơi giữ vị trí đọc/ghi hiện tại và các cờ trạng thái. Hai descriptor có thể cùng tham chiếu một open file description; khi đó chúng chia sẻ vị trí đọc/ghi.

Sự khác biệt có ý nghĩa khi đọc log. Nếu một tiến trình đã mở tệp log và người vận hành đổi tên hoặc xóa đường dẫn, descriptor đang mở vẫn có thể trỏ tới đối tượng cũ. Tạo một tệp mới ở cùng tên đường dẫn không buộc tiến trình chuyển đầu ghi sang tệp mới. Phần thực hành về log rotation ở các chương sau sẽ dựa trên quan hệ này để giải thích vì sao có trường hợp dung lượng đĩa chưa được giải phóng và vì sao cần yêu cầu dịch vụ mở lại log.

Ba descriptor quen thuộc khi khởi chạy chương trình là đầu vào chuẩn, đầu ra chuẩn và đầu lỗi chuẩn. Chuyển hướng trong shell thay đổi các luồng mà tiến trình nhận được; nó không sửa ý nghĩa của chương trình. Khi một dịch vụ ghi thông báo lỗi vào đầu ra chuẩn, người thu thập log vẫn có thể lấy được dữ liệu, nhưng việc phân loại mức độ lỗi trở nên khó hơn. Do vậy, quy ước đầu ra là một phần của hợp đồng vận hành.

1.4. Quyền và giới hạn: tiến trình không có toàn quyền chỉ vì nó chạy

Một tiến trình mang thông tin định danh và chịu kiểm tra quyền khi truy cập tài nguyên. Có quyền đọc một đường dẫn không đồng nghĩa có quyền thực thi tệp, và có quyền thực thi một tệp không đồng nghĩa được đọc mọi tệp mà chương trình sẽ mở sau đó. Nhìn từ bên ngoài, cả hai tình huống có thể hiện thành "ứng dụng không khởi động"; bên trong, điểm lỗi nằm ở những bước khác nhau. Về sau, mô hình này sẽ nối với quyền Linux, IAM và quyền của tài khoản dịch vụ trong Kubernetes.

Tiến trình còn hoạt động trong các giới hạn tài nguyên. Một lần tạo tiến trình con có thể thất bại khi đạt giới hạn số tác vụ; `fork` báo lỗi thay vì bảo đảm thành công. Vì vậy, một bộ điều phối không thể chữa một chương trình tạo tiến trình vô hạn chỉ bằng cách tự khởi động lại nó. Tự phục hồi là chính sách phản ứng với lỗi; ngăn lỗi lặp lại đòi sửa nguyên nhân hoặc đặt giới hạn thích hợp.

Lưu ý rằng quyền truy cập và khả năng nhìn thấy tài nguyên là hai câu hỏi khác nhau. Một namespace có thể cho tiến trình một góc nhìn riêng về một loại tài nguyên của hệ thống. Nhưng góc nhìn riêng không tự chứng minh rằng tiến trình an toàn hay không thể tác động đến thứ bên ngoài ranh giới đó. Ở phần container, ta sẽ kết hợp namespace với giới hạn tài nguyên, quyền và chính sách bảo mật thay vì xem "cách ly" là một thuộc tính nhị phân.

1.5. Từ tiến trình đến dịch vụ

Dịch vụ là một hợp đồng vận hành đặt lên tiến trình: khởi động khi nào, cấu hình từ đâu, báo trạng thái ra sao, dừng theo tín hiệu nào, có khởi động lại không và dữ liệu bền vững nằm ở đâu. Không phải mọi tiến trình đều là dịch vụ. Một công cụ chạy xong rồi thoát có vòng đời khác một HTTP server tồn tại lâu và tiếp nhận yêu cầu mới.

Khi một tiến trình thoát, hệ điều hành có thể ghi nhận mã kết thúc. Mã này cho biết chương trình kết thúc theo quy ước nào; tự nó chưa cho biết nguyên nhân gốc. Cùng một mã khác không có bảo đảm diễn giải giống nhau giữa mọi chương trình. Một hệ thống giám sát tốt cần xem thêm thời điểm, stderr, tình trạng tài nguyên và sự kiện từ bộ quản lý dịch vụ. Đọc mỗi trạng thái "đã dừng" rồi suy ra lỗi mạng, lỗi quyền hay thiếu bộ nhớ là bước nhảy thiếu chứng cứ.

Một dịch vụ cũng có thể còn tiến trình nhưng không còn khả năng làm việc có ích. Vòng lặp bị kẹt, hàng đợi đầy hoặc phụ thuộc phía sau không phản hồi đều có thể khiến PID vẫn tồn tại. Vì vậy, giám sát sự sống của tiến trình và kiểm tra khả năng phục vụ là hai phép đo khác nhau. Ta sẽ gặp lại khác biệt này khi phân tích healthcheck của Docker, probe của Kubernetes và SLI của SRE.

1.6. Áp mô hình vào một sự cố nhỏ

Giả sử sau triển khai, trình duyệt báo không kết nối được tới dịch vụ. Cách suy nghĩ theo lớp bắt đầu từ artifact và tiến trình: bản build đã đến máy chưa; tiến trình có được tạo không; nó còn tồn tại hay đã thoát? Nếu đã thoát, cần xem mã kết thúc và thông báo lỗi. Nếu còn tồn tại, xem nó đã tạo socket lắng nghe đúng địa chỉ và cổng chưa. Sau đó mới xem đường đi mạng và phụ thuộc ở xa. Trật tự này không bảo đảm tìm ra mọi lỗi, nhưng ngăn việc sửa DNS khi chương trình thực ra chưa mở cổng.

Khi kiểm tra, phải phân biệt quan sát với kết luận. “Không thấy PID trong danh sách” là quan sát tại thời điểm đo; “build lỗi” chỉ là một giả thuyết. “Có PID” cũng chưa chứng minh dịch vụ khỏe. Một hồ sơ sự cố tốt ghi mốc thời gian, lệnh hoặc nguồn quan sát, artifact đang chạy, cấu hình có hiệu lực và thay đổi vừa xảy ra. Nhờ thế, người khác có thể phản biện kết luận hoặc tái dựng chuỗi nhân quả thay vì tin vào trí nhớ của người trực hệ thống.

Chương này không yêu cầu thuộc lòng mọi lời gọi hệ thống. Điều cần giữ là ba câu hỏi: thứ gì tồn tại trên đĩa, tiến trình nào đang chạy, và tiến trình ấy đang nắm những tài nguyên nào? Phần sau sẽ dùng cùng ba câu hỏi khi đi từ một tiến trình trên máy tới container, rồi tới workload trong cụm.

Bài tập

C1.1. Một bản triển khai thay tệp thực thi trên đĩa nhưng không khởi động lại dịch vụ. Giải thích vì sao không được kết luận tiến trình đang phục vụ đã chạy phiên bản mới. Nêu phép kiểm tra cần làm.

C1.2. Một chương trình vẫn ghi vào tệp log đã bị đổi tên; tệp mới cùng tên vẫn rỗng. Hãy dùng quan hệ giữa đường dẫn, descriptor và open file description để giải thích. Đề xuất một cách xử lý an toàn.

C1.3. `fork` thành công, `execve` thất bại. Trong mô hình của chương này, có bao nhiêu tiến trình sau `fork`, và tiến trình nào có thể nhận biết lỗi `execve`? Vì sao không được coi `execve` là thao tác tạo tiến trình?

Chương 2. CPU, bộ nhớ và I/O: thời gian đi đâu?

Một yêu cầu HTTP mất hai giây không có nghĩa CPU đã làm việc trong hai giây. Trong khoảng ấy, tiến trình có thể đợi lịch chạy, đợi ổ lưu trữ, đợi mạng hoặc chờ khóa do chính chương trình giữ. Nếu không phân biệt những khoảng thời gian này, ta rất dễ tăng số CPU cho một dịch vụ đang nghẽn ở dữ liệu, hoặc tăng bộ nhớ cho một tiến trình đang bị giới hạn thời gian CPU. Chương này xây dựng một cách phân rã độ trễ từ máy tính trước khi dùng công cụ giám sát.

2.1. Công việc, thời gian chạy và thời gian chờ

CPU chỉ thực hiện lệnh của một luồng tại một thời điểm trên mỗi đơn vị xử lý logic. Khi nhiều luồng cùng sẵn sàng, bộ lập lịch chọn luồng nào được chạy tiếp. Một luồng có thể đang chạy, sẵn sàng nhưng chưa được chọn, hoặc không sẵn sàng vì đang chờ một sự kiện. Trạng thái "chờ" không đồng nhất: chờ dữ liệu mạng khác với chờ khóa, và cả hai khác với việc sẵn sàng nhưng bị giới hạn quota CPU. Những lý do này đòi cách sửa khác nhau.

Hãy xét một ví dụ tính toán, không phải kết quả đo: yêu cầu A hoàn tất sau 100 ms theo đồng hồ, trong đó CPU xử lý 40 ms và các khoảng chờ cộng lại 60 ms. Độ trễ của A là 100 ms, còn CPU time gắn với phần xử lý của A là 40 ms. Nếu chỉ giảm thời gian CPU một nửa mà giữ nguyên mọi khoảng chờ, độ trễ mới tốt nhất trong mô hình đơn giản này là 80 ms. Kết quả đó không tiên đoán hệ thống thực; nó chỉ cho thấy không thể lấy tỷ lệ CPU bận để suy trực tiếp độ trễ của từng yêu cầu.

Thông lượng là số công việc hoàn thành trong một khoảng thời gian; độ trễ là thời gian một công việc mất từ đầu đến cuối. Hai đại lượng liên hệ nhưng không thay thế nhau. Một hệ thống có thể giữ thông lượng bằng cách xếp thêm yêu cầu vào hàng đợi, trong khi người dùng chờ lâu hơn. Vì thế, khi nói "cần thêm máy", phải biết mục tiêu là tăng thông lượng, giảm độ trễ ở một mức tải hay giữ hệ thống ổn định khi có đột biến.

Lập lịch công bằng là một mục tiêu chính sách, không phải lời hứa mỗi tiến trình luôn nhận đúng một phần CPU cố định. Chính sách, độ ưu tiên, số tác vụ sẵn sàng, ràng buộc nhóm và phiên bản nhân đều ảnh hưởng kết quả. Tài liệu Linux mô tả quá trình chuyển từ CFS sang EEVDF ở các nhân mới; cuốn sách này giữ mô hình chung "tác vụ sẵn sàng được chọn theo chính sách" ở phần nền tảng, và chỉ bàn thuật toán cụ thể khi có phiên bản nhân làm mốc.

2.2. Địa chỉ ảo không phải RAM đã dùng

Mỗi tiến trình nhìn thấy một không gian địa chỉ ảo. Chương trình dùng các địa chỉ trong không gian ấy; phần cứng và nhân phối hợp ánh xạ chúng tới trang bộ nhớ vật lý khi cần. Hai tiến trình có thể dùng cùng một giá trị địa chỉ ảo mà không mặc nhiên đọc cùng dữ liệu. Điều này tạo ranh giới bảo vệ và cho phép quản lý bộ nhớ linh hoạt hơn một sơ đồ "biến nằm ở ô RAM cố định".

Một vùng địa chỉ được dành cho tiến trình chưa nhất thiết đã có một trang RAM riêng. Chẳng hạn, vùng ánh xạ ẩn danh có thể được tạo trước, còn trang vật lý được cấp khi truy cập hoặc ghi phù hợp. Khi một địa chỉ chưa có ánh xạ vật lý cần thiết, phần cứng báo page fault để nhân xử lý. Page fault

không tự đồng nghĩa lỗi chương trình: nhiều fault là một phần của cấp phát theo nhu cầu. Lỗi thật xuất hiện khi truy cập không hợp lệ hoặc không thể đáp ứng điều kiện ánh xạ.

Vì vậy, một con số "bộ nhớ ảo" lớn không đủ để kết luận sắp hết RAM. Ngược lại, nhìn một con số bộ nhớ của riêng tiến trình cũng chưa cho biết áp lực bộ nhớ toàn máy: trang có thể được chia sẻ, nằm trong cache, hoặc chịu quy tắc tính của nhóm tài nguyên. Khi chẩn đoán, cần xác định phép đo đang đếm địa chỉ, trang cư trú, trang dùng chung hay bộ nhớ tính cho một cgroup. Nhầm tên đại lượng dẫn tới tăng giới hạn sai chỗ.

2.3. Page cache và lời hứa ghi dữ liệu

Đọc một tệp thường đi qua page cache của Linux. Dữ liệu vừa đọc có thể được giữ trong RAM để lần truy cập sau không phải lấy lại từ thiết bị. Cache vì thế không đơn giản là "RAM bị chiếm vô ích"; nó là một cách đổi bộ nhớ lấy độ trễ I/O thấp hơn. Khi thiếu bộ nhớ, nhân có thể thu hồi những trang phù hợp. Tuy nhiên, khả năng thu hồi và chi phí thu hồi khác nhau giữa trang sạch của tệp, trang đã sửa nhưng chưa ghi xuống thiết bị, và vùng nhớ ẩn danh.

Ghi vào tệp cũng có thể hoàn tất ở tầng lời gọi ứng dụng trước khi mọi thay đổi đã nằm trên thiết bị bền vững. Trang đã sửa trong cache được đánh dấu dirty và sau đó cần được đồng bộ. Vì vậy, "lời gọi ghi trả về" và "dữ liệu sống sót qua mất điện" là hai hợp đồng khác nhau. Trên Linux, `fsync` yêu cầu đồng bộ dữ liệu cùng metadata của tệp tới thiết bị theo hợp đồng của lời gọi; khi tạo hoặc đổi tên tệp, việc đồng bộ riêng descriptor của tệp chưa nhất thiết bảo đảm mục thư mục chứa tên đó đã bền vững. Ứng dụng có dữ liệu quan trọng phải xác định chính xác mình cần bảo đảm điều gì, thay vì gọi chung là "đã lưu".

Một thao tác sao lưu an toàn vì thế không chỉ là sao chép bytes rồi thấy lệnh trả 0. Cần kiểm tra nhất quán dữ liệu đầu vào, kết quả ghi, ranh giới bền vững cần thiết và khả năng phục hồi. Hệ thống tệp, cơ sở dữ liệu và object storage cung cấp các hợp đồng khác nhau; chương về lưu trữ sẽ đi sâu vào từng loại. Ở đây điều cần nhớ là I/O có nhiều tầng và độ trễ ở từng tầng không hiện ra từ một phép đo CPU.

2.4. Giới hạn tài nguyên và hành vi dưới tải

Trong môi trường container, cgroup là một cơ chế Linux dùng để tổ chức và kiểm soát tài nguyên của nhóm tác vụ. Một giới hạn CPU có thể khiến nhóm bị trì hoãn khi dùng hết quota trong chu kỳ; một giới hạn bộ nhớ có thể làm tăng thu hồi và cuối cùng dẫn tới lỗi cấp phát hoặc tiến trình bị kết thúc theo chính sách. Hai tình huống này có thể cùng biểu hiện thành tăng độ trễ, nhưng dấu vết và cách xử lý khác nhau. Không nên gọi mọi trường hợp "container chậm" là thiếu CPU.

Để phân biệt, hãy đặt câu hỏi theo chuỗi. Công việc có sẵn sàng chạy nhưng không nhận CPU không? Bộ nhớ nào đang tăng, và đó là cache có thể thu hồi hay dữ liệu hoạt động? Có thao tác I/O nào bị chặn? Giới hạn nào áp cho tiến trình, container hoặc cả máy? Nếu quan sát cho thấy CPU bị giới hạn theo quota, tăng số replica có thể chia tải, nhưng cũng có thể làm áp lực lên phụ thuộc chung. Nếu quan sát cho thấy ứng dụng chờ cơ sở dữ liệu, thêm CPU cho nó không loại bỏ thời gian

chờ ấy.

Những câu hỏi trên là mô hình phân tích, không phải một runbook tự động. Chúng chuẩn bị cho các chương sau về đo lường, HPA và SLO. Trước khi dùng một metric để tự thay đổi số bản sao, phải biết metric đại diện cho tài nguyên nào, cửa sổ thời gian nào và điều gì xảy ra khi phép đo bị thiếu. Một bộ điều khiển ra quyết định từ tín hiệu sai vẫn có thể hoạt động đúng theo thuật toán của nó nhưng tạo kết quả vận hành xấu.

2.5. Lần theo một phút tăng độ trễ

Xét một dịch vụ đọc tệp trong một cgroup v2. Bảng sau là dữ liệu giả định để tập tính và phản biện, không phải đầu ra đo từ máy hay bằng chứng cho một sự cố có thật. Hai cửa sổ đều dài 60 giây, có cùng số yêu cầu và cùng giới hạn tài nguyên. Các giá trị `usage_usec`, `throttled_usec` và `io.pressure some total` là mức tăng của counter tích lũy giữa đầu và cuối mỗi cửa sổ; `memory.current` là giá trị ở cuối cửa sổ. Người đo phải lấy đúng cgroup chứa workload, cùng ranh giới thời gian với số liệu yêu cầu, và kiểm tra các tệp này có sẵn trên nhân đang chạy.

Quan sát trong 60 giây	Bình thường	Độ trễ tăng
p95 yêu cầu	120 ms	870 ms
`cpu.stat usage_usec` tăng	18.000.000 µs	19.200.000 µs
`cpu.stat nr_throttled` tăng	0	0
`memory.current` cuối kỳ	310 MiB	315 MiB
`memory.events high / oom` tăng	0 / 0	0 / 0
`io.pressure some total` tăng	1.200.000 µs	18.000.000 µs

Chia CPU time cho 60 giây cho ta mức dùng trung bình 0,30 rồi 0,32 lõi, không phải 30% rồi 32% độ trễ của từng yêu cầu. `nr_throttled` không tăng nên trong hai cửa sổ này chưa có dấu hiệu cgroup đó bị giới hạn bởi quota CPU; việc luồng phải đợi lịch chạy vì tác vụ khác vẫn có thể xảy ra. Bộ nhớ cuối kỳ tăng 5 MiB và không có sự kiện `high` hay `oom`, nên không có dấu vết vượt hai ngưỡng ấy trong counter đã xem; điều đó không loại trừ thu hồi bộ nhớ toàn máy, page fault hoặc một ngưỡng ở cgroup tổ tiên. Muốn xét áp lực bộ nhớ, đọc thêm `memory.stat`, `memory.pressure` và giới hạn thực tế theo cùng cửa sổ.

Với I/O, `some total` tăng 1,2 giây rồi 18 giây: lần lượt khoảng 2% và 30% của mỗi cửa sổ có ít nhất một tác vụ trong nhóm bị nghẽn I/O. Đây là dấu vết mạnh hơn cho giả thuyết chờ I/O trong cửa sổ chậm, nhưng nó không nói yêu cầu nào chờ, chờ thiết bị nào, hay I/O là nguyên nhân duy nhất của p95 tăng. Bước kế tiếp là ghép thời điểm yêu cầu chậm với thời gian thao tác đọc, bộ đếm bytes/thao tác và tình trạng thiết bị hoặc phụ thuộc tương ứng; cũng phải xét khóa trong ứng dụng và mạng. Nếu đó là đọc từ page cache, số bytes ở thiết bị có thể không phản ánh số lần ứng dụng đọc. Kết luận đúng ở giai đoạn này là ưu tiên kiểm tra đường I/O, chưa phải "ổ đĩa hỏng" hoặc "thêm CPU sẽ hết chậm".

Bài tập

C2.1. Trong một ví dụ giả định, độ trễ yêu cầu là 100 ms gồm 40 ms CPU và 60 ms chờ. Nếu chỉ giảm phần CPU xuống 20 ms, độ trễ thấp nhất theo mô hình cộng đơn giản là bao nhiêu? Nêu một lý do khiến kết quả thực tế khác phép tính.

C2.2. Một tiến trình có không gian địa chỉ ảo lớn nhưng bộ nhớ cư trú thấp. Vì sao chưa thể kết luận nó đang chiếm gần hết RAM? Cần phân biệt ít nhất hai đại lượng nào?

C2.3. Dịch vụ ghi một tệp cấu hình mới rồi đổi tên tệp để thay bản cũ. Việc lời gọi ghi trả về thành công đã đủ chứng minh tên tệp mới sống sót sau mất điện chưa? Giải thích ranh giới cần kiểm tra.

C2.4. Dùng số liệu giả định ở mục 2.5: tính mức dùng CPU trung bình và tỷ phần thời gian `io.pressure some` của hai cửa sổ; nêu dấu vết cho hoặc chống lại ba giả thuyết quota CPU, giới hạn bộ nhớ và chờ I/O. Chỉ ra một điều bảng chưa thể kết luận và phép đo kế tiếp.

Chương 3. Nhân Linux, hệ thống tệp, quyền và systemd

Một dịch vụ có thể chạy dưới đúng người dùng mà vẫn không đọc được cấu hình. Một tệp có bit đọc nhưng đường dẫn tới nó lại không thể đi qua. Một lệnh `systemctl start` có thể trả thành công trong khi ứng dụng chưa sẵn sàng nhận yêu cầu. Những hiện tượng này thuộc các lớp khác nhau: nhân kiểm tra lời gọi hệ thống và quyền truy cập, hệ thống tệp ánh xạ tên sang đối tượng, còn systemd điều hành vòng đời tiến trình. Chương này nối các lớp đó qua một trường hợp vận hành có thể tự kiểm chứng.

3.1. Từ mã người dùng đến lời gọi hệ thống

Tiến trình thực thi phần lớn mã của nó ở user space. Khi cần mở tệp, tạo socket hay đổi quyền, nó gọi một giao diện mà cuối cùng đi vào nhân. Ví dụ `openat` nhận đường dẫn, cờ mở và ngữ cảnh của tiến trình; nhân phân giải đường dẫn, kiểm tra quyền, rồi trả về file descriptor hoặc lỗi. Một hàm thư viện hay một lệnh shell có thể gọi thêm nhiều bước trước và sau lời gọi này. Vì vậy, thông báo “Permission denied” của ứng dụng cần được lần tới thao tác thực sự thất bại; nó không tự cho biết bit quyền nào sai.

Nhân là nơi thực thi các quy tắc về tiến trình, bộ nhớ, đối tượng tệp và thiết bị. systemd là một tiến trình ở user space dùng các giao diện ấy để khởi động và giám sát dịch vụ. `systemctl` lại là công cụ yêu cầu systemd thực hiện thao tác. Ba tên này không thay thế nhau: nhân không đọc mục `Restart=` trong unit, còn systemd không bỏ qua được phép kiểm tra của nhân chỉ vì unit đã nạp thành công.

Sau một lời gọi thất bại, chương trình có thể nhận `EACCES` khi thiếu quyền tìm kiếm hoặc truy cập, `ENOENT` khi thành phần đường dẫn không tồn tại, hay `ENOTDIR` khi một thành phần trung gian không phải thư mục. Đó là mã lỗi ở ranh giới lời gọi; ứng dụng có thể gộp hoặc đổi thông báo khi hiển thị. Trước khi sửa quyền, phải xác định đường dẫn thực, người dùng hiệu lực và thao tác đọc, ghi hay thực thi mà tiến trình đã yêu cầu.

3.2. Tên tệp, inode và bước đi qua thư mục

Đường dẫn là chuỗi các thành phần được tra cứu từ một điểm bắt đầu. Mỗi thư mục tham gia phép tra cứu tên; tệp cuối cùng có metadata như loại, quyền và số inode. Tên không đồng nhất với inode: đổi tên thay quan hệ trong thư mục, còn một descriptor đang mở vẫn tham chiếu đối tượng đã mở như Chương 1 đã giải thích. Một hard link khác có thể trỏ tới cùng inode trên cùng hệ thống tệp, còn symbolic link chứa một đường dẫn cần được phân giải tiếp. Vì thế, xem riêng `ls -l` của tệp đích có thể bỏ sót vấn đề ở thư mục cha hoặc một liên kết trung gian.

Với chế độ quyền Linux thông thường, mỗi nhóm bit đọc, ghi, thực thi áp cho chủ sở hữu, nhóm và người khác. Trên tệp thường, bit `x` cho phép thực thi theo các điều kiện của định dạng tệp; trên thư mục, bit `x` là quyền tìm kiếm/đi qua thành phần tên, không phải “chạy thư mục”. Bit `r` của thư mục cho phép liệt kê tên; biết sẵn một tên và tra cứu nó cần quyền tìm kiếm. Khi tạo hoặc xóa tên trong

thư mục, quyền ở thư mục có vai trò riêng, không thể suy chỉ từ bit ghi trên tệp. ACL, mount option, namespace, chính sách bảo mật bổ sung và quyền đặc biệt có thể làm kết quả khác mô hình ba bộ bit cơ bản; mô hình đó là điểm bắt đầu để khoanh vùng, không phải toàn bộ chính sách.

Xét cây `/srv/atlas/config/app.toml`: `app.toml` là `0640`, thuộc `root:atlas`; tiến trình chạy với nhóm `atlas`. Tệp có vẻ đọc được qua bit nhóm, nhưng nếu `/srv/atlas` là `0700 root:root`, tiến trình không thể đi qua thư mục này và thao tác mở có thể trả `EACCES`. Cách sửa đúng phụ thuộc hợp đồng truy cập: có thể cấp quyền tìm kiếm tối thiểu trên thư mục cho nhóm phù hợp, hoặc đặt cấu hình ở một cây do dịch vụ được phép đi qua. Đổi `app.toml` thành `0666` không sửa được đường đi, lại mở rộng người có thể đọc và sửa bí mật trong tệp. Không dùng `chmod -R 777` như cách chẩn đoán.

Để kiểm chứng trên máy Linux thử nghiệm do mình kiểm soát, ghi nhận danh tính tiến trình bằng `id`, xem từng thành phần bằng `namei -l /srv/atlas/config/app.toml`, rồi thử đọc dưới đúng tài khoản dịch vụ, chẳng hạn `sudo -u atlas cat /srv/atlas/config/app.toml`. Trước khi thay quyền, xác nhận các đường dẫn và tài khoản thực sự tồn tại; các tên trong ví dụ là giả định. `namei` giúp nhìn chuỗi quyền nhưng kết quả đọc cuối cùng mới kiểm tra đúng tổ hợp tài khoản, nhóm, ACL và mount đang có hiệu lực. Nếu unit cấp thêm sandbox hoặc thay root directory, phải kiểm tra từ ngữ cảnh của dịch vụ thay vì chỉ dùng một shell bên ngoài.

3.3. Một unit biến tiến trình thành dịch vụ như thế nào

Một unit service mô tả chương trình chính, danh tính chạy, thứ tự phụ thuộc và chính sách khi nó kết thúc. Ví dụ minh họa sau dùng đường dẫn giả định; trước khi cài phải tạo tài khoản, triển khai chương trình và cấu hình đúng trên máy đích. `Type=exec` khiến thao tác khởi động chờ tới khi thực thi được binary; nó bắt được trường hợp executable hoặc user không tồn tại tốt hơn `Type=simple`. Nó vẫn không chứng minh mã ứng dụng đã đọc xong cấu hình, mở socket hoặc sẵn sàng phục vụ.

```
[Unit]
Description=Atlas example service
After=network.target

[Service]
Type=exec
User=atlas
Group=atlas
WorkingDirectory=/opt/atlas
ExecStart=/opt/atlas/bin/atlas --config /srv/atlas/config/app.toml
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

Trong ví dụ, systemd tạo tiến trình với `User=atlas` và `Group=atlas`, đặt thư mục làm việc ở `/opt/atlas`, rồi thực thi chương trình được chỉ ra ở `ExecStart`. Thư mục làm việc tách khỏi cây cấu hình để lỗi quyền ở `/srv/atlas` xuất hiện khi ứng dụng gọi `openat`, sau bước khởi chạy. Khi ấy nhân vẫn kiểm tra đường dẫn và quyền theo ngữ cảnh đã thiết lập. `Restart=on-failure` yêu

cầu thử chạy lại khi tiến trình kết thúc theo điều kiện lỗi được systemd định nghĩa; nó không chữa lỗi quyền cố định. `RestartSec=5s` chỉ tạo khoảng chờ giữa các lần thử và giới hạn tần suất khởi động vẫn có thể chặn vòng lặp lỗi.

Đường lỗi có thể lần theo như sau. Nếu `/opt/atlas/bin/atlas` không tồn tại, bước thực thi thất bại và `Type=exec` báo lỗi khởi động. Nếu binary chạy được nhưng không đi qua `/srv/atlas`, chương trình nhận lỗi khi mở cấu hình và tự thoát với mã lỗi; systemd ghi nhận tiến trình chính đã thoát, áp chính sách restart, rồi có thể chạm giới hạn khởi động. Nếu binary còn chạy nhưng kẹt ở phụ thuộc khác, trạng thái `active` chỉ nói tiến trình được quản lý còn ở trạng thái ấy; nó không phải healthcheck nghiệp vụ. Mỗi đường lỗi cần xem `systemctl status atlas.service`, `journalctl -u atlas.service` và phép thử khả năng phục vụ của chính ứng dụng. Không lấy riêng trạng thái `active` để tuyên bố đã triển khai thành công.

Khi yêu cầu dừng, systemd gửi tín hiệu theo cấu hình dừng và chờ tiến trình kết thúc trong thời hạn quy định; nếu tiến trình không dừng, chính sách có thể dẫn tới tín hiệu cưỡng bức. Ứng dụng phải xử lý tín hiệu phù hợp, ngừng nhận công việc mới, hoàn tất hoặc hủy công việc đang làm theo hợp đồng, đóng tài nguyên và trả mã kết thúc rõ ràng. `Restart=` mô tả phản ứng sau khi tiến trình mất; nó không bảo đảm dữ liệu được ghi bền vững hoặc yêu cầu đang xử lý được giữ lại. Muốn có ranh giới an toàn, phải thiết kế riêng giao thức dừng và kiểm tra nó dưới tải.

3.4. Kiểm chứng trước và sau khi khởi động

Unit có thể được kiểm tra cú pháp và một phần tham chiếu bằng `systemd-analyze verify` trên tệp unit. Sau khi cài unit vào nơi systemd đọc và chạy `daemon-reload`, cần kiểm tra user/group và đường dẫn đã tồn tại, rồi mới khởi động. Trên máy đang quản lý dịch vụ, quan sát `systemctl show` các thuộc tính như `MainPID`, `ExecMainStatus`, `ActiveState` và `SubState` giúp phân biệt “chưa tạo được process”, “process thoát” và “process còn sống”. Nhật ký đặt các bước vào trục thời gian; cùng một mã thoát chỉ là dấu vết, không phải lời giải thích nguyên nhân.

Một kiểm tra nhỏ nhưng có sức phân biệt là chủ động gây lỗi trong môi trường thử nghiệm: tạm trỏ unit thử tới một binary không tồn tại để quan sát lỗi ở bước exec, rồi trả lại; tiếp đó thử một đường dẫn cấu hình mà tài khoản dịch vụ không đi qua được để quan sát lỗi sau khi process đã khởi chạy. Hai phép thử không nên làm trên dịch vụ production. Chúng dạy cách đọc ranh giới: trường hợp đầu thuộc thiết lập tiến trình của systemd, trường hợp sau là lời gọi tệp trong ứng dụng bị nhân từ chối. Sau mỗi thay đổi phải hoàn nguyên cấu hình thử, kiểm tra trạng thái và dữ liệu; không dùng restart lặp vô hạn làm bằng chứng tự phục hồi.

Ở điểm này, một dịch vụ có bốn câu hỏi kiểm chứng khác nhau: artifact có ở đúng đường dẫn không; tiến trình có được tạo dưới đúng danh tính không; các lời gọi tài nguyên của nó có thành công không; và người dùng có nhận được chức năng mong đợi không. Chúng lần lượt nối Chương 1, Chương 2 và chương này. Phần mạng sẽ bổ sung đường đi của yêu cầu qua địa chỉ và giao thức, nơi một PID khỏe vẫn có thể không phục vụ được người dùng ở xa.

Bài tập

C3.1. Tệp cấu hình có mode `0640 root:atlas`, tiến trình thuộc nhóm `atlas`, nhưng thư mục cha `/srv/atlas` có mode `0700 root:root`. Vì sao thao tác đọc có thể bị từ chối? Nêu một thay đổi tối thiểu cần kiểm tra và một thay đổi không nên dùng.

C3.2. Với `Type=exec`, `systemctl start` thành công. Điều này xác nhận được bước nào và chưa xác nhận được bước nào của một HTTP service? Nêu hai quan sát tiếp theo.

C3.3. Phân biệt hai lỗi: binary trong `ExecStart` không tồn tại và binary chạy được nhưng `openat` cấu hình trả `EACCES`. Thành phần nào ghi nhận lỗi đầu tiên, systemd có thể phản ứng ra sao, và vì sao tăng `RestartSec` không giải quyết nguyên nhân?

Lời giải

Chương 1. Từ chương trình đến tiến trình

C1.1. Artifact trên đĩa và lần thực thi đang chạy là hai thực thể khác nhau. Việc ghi đè một đường dẫn không chứng minh process image đã đổi. Cần kiểm tra PID và thời điểm khởi động, lệnh và artifact mà dịch vụ đang dùng, rồi so với định danh bản triển khai. Nếu ứng dụng có endpoint phiên bản, chỉ dùng nó khi biết endpoint phản ánh đúng build đang chạy. Khởi động lại phải theo quy trình triển khai, có kiểm tra sức khỏe và đường quay lui.

C1.2. Descriptor của tiến trình tham chiếu open file description đang mở, không tra cứu lại tên đường dẫn cho mỗi lần ghi. Đổi tên hoặc tạo tệp mới cùng tên không chuyển descriptor cũ. Cách xử lý là yêu cầu dịch vụ mở lại log bằng cơ chế được nó hỗ trợ, hoặc khởi động lại có kiểm soát sau khi bảo đảm log đã được thu thập và khả năng phục vụ không bị gián đoạn ngoài mức chấp nhận. Không xóa cưỡng bức dữ liệu trước khi xác định descriptor và nhu cầu lưu giữ.

C1.3. Sau `fork` thành công có hai tiến trình: cha và con. Nếu con gọi `execve` và lời gọi thất bại, chính nhánh con nhận giá trị lỗi và có thể báo lại; cha chỉ biết nếu có cơ chế thông báo hoặc chờ kết thúc. `execve` thay chương trình trong tiến trình gọi, nên không tăng số tiến trình. Trên đường thành công, lệnh sau `execve` trong chương trình cũ không chạy tiếp.

Chương 2. CPU, bộ nhớ và I/O: thời gian đi đâu?

C2.1. Theo mô hình cộng đã cho, độ trễ còn 20 + 60 = 80 ms. Trên hệ thống thật, thay đổi thời gian CPU có thể đổi lịch chạy, hành vi cache, mức cạnh tranh hoặc thời gian chờ của các yêu cầu khác; 80 ms chỉ là kết quả của giả định giữ mọi yếu tố kia cố định.

C2.2. Không gian địa chỉ ảo đo vùng địa chỉ mà tiến trình có thể dùng, không phải số trang vật lý đang cư trú. Cần phân biệt kích thước ánh xạ ảo và bộ nhớ cư trú; nếu phân tích áp lực chung, còn phải xét trang dùng chung, cache và cách cgroup tính bộ nhớ.

C2.3. Chưa đủ. Lời gọi ghi thành công không tự bảo đảm dữ liệu và metadata đã tới thiết bị bền vững. Khi tên tệp được tạo hoặc đổi, cần kiểm tra hợp đồng đồng bộ tệp và thư mục chứa mục tên đó, cùng hành vi hệ thống tệp thực tế. Không nên biến `fsync` trên tệp thành lời hứa tuyệt đối cho mọi bước của thao tác thay tên.

C2.4. `cpu.stat usage_usec` chia cho 60.000.000 µs cho 0,30 và 0,32 lõi trung bình; `nr_throttled` không tăng nên chưa có dấu vết quota CPU tại cgroup đã đo. `memory.current` cuối kỳ tăng 5 MiB, còn `memory.events high` và `oom` không tăng; chưa thấy chạm hai ngưỡng này, nhưng chưa loại trừ thu hồi ở máy hoặc cgroup tổ tiên. `io.pressure some total` chia cho 60.000.000 µs cho 2% và 30% thời gian có ít nhất một tác vụ nghẽn I/O. Cùng với p95 tăng, đó là lý do ưu tiên điều tra I/O. PSI không gắn thời gian chờ với từng yêu cầu hay thiết bị; bước tiếp theo là đo thời gian thao tác đọc theo yêu cầu, đối chiếu `io.stat` và tình trạng thiết bị trong cùng cửa sổ. Chưa thể quy toàn bộ độ trễ cho I/O.

Chương 3. Nhân Linux, hệ thống tệp, quyền và systemd

C3.1. Nhóm `atlas` có thể đọc tệp theo bit nhóm của `0640`, nhưng không có quyền tìm kiếm trên `/srv/atlas` khi thư mục là `0700 root:root`; phân giải đường dẫn có thể dừng ở thư mục đó với `EACCES`. Kiểm tra quyền từng thành phần và danh tính thực của dịch vụ; nếu hợp đồng cho phép, cấp quyền tìm kiếm trên thư mục cho đúng nhóm hoặc chuyển cấu hình sang cây thích hợp. Không đổi tệp thành `0666` hay dùng `chmod -R 777`: hai cách đó vừa không sửa quyền đi qua thư mục, vừa mở rộng truy cập.

C3.2. `Type=exec` xác nhận systemd đã thiết lập và thực thi được binary chính, theo hợp đồng của loại unit này. Nó không chứng minh ứng dụng đọc cấu hình thành công, mở cổng hay trả HTTP đúng. Xem nhật ký và trạng thái tiến trình chính, rồi gọi health endpoint hoặc phép thử chức năng từ đúng đường đi của người dùng. Một PID `active` không đủ thay cho phép thử khả năng phục vụ.

C3.3. Với binary thiếu, bước chuẩn bị/thực thi của systemd thất bại trước khi mã ứng dụng chạy. Với `openat` trả `EACCES`, binary đã chạy; nhân từ chối một thao tác tệp của tiến trình, ứng dụng phải ghi lỗi và chọn cách thoát. Nếu tiến trình thoát lỗi, `Restart=on-failure` có thể thử lại, rồi bị giới hạn tần suất khởi động. Tăng `RestartSec` chỉ đổi nhịp thử; đường dẫn hoặc quyền sai vẫn sai. Cần sửa đúng artifact, danh tính và quyền sau khi kiểm tra nguyên nhân.